React Native for Beginners

A hands-on introduction to building mobile apps with React Native. By the end, you'll have built a working app and understand the core concepts.

What You'll Need

- Node.js installed (v18 or newer)
- A code editor (VS Code recommended)
- A phone with the **Expo Go** app, or a simulator/emulator on your computer

We'll use **Expo**, the easiest way to start with React Native — no need to install Android Studio or Xcode just to get going.

1. Create Your First App

Open a terminal and run:

```
npx create-expo-app@latest MyFirstApp
cd MyFirstApp
npx expo start
```

A QR code will appear in your terminal. Scan it with the Expo Go app on your phone (Camera app on iOS, Expo Go app on Android) and your app will load live on your device.

2. Understanding the Basics

Open `App.js`. You'll see something like this:

```
import { StyleSheet, Text, View } from 'react-
native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello, world!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    alignItems: 'center',
    justifyContent: 'center',
  },
});
```

Key ideas here:

- `View` is like a `<div>` — a container for layout.
- `Text` is required to display any text. Unlike web, you can't just drop text inside a `View`.
- `StyleSheet.create` defines styles using JavaScript objects (camelCase properties, no CSS files).
- **Flexbox** is the default layout system. `flex: 1` means "take up all available space."

Try changing `"Hello, world!"` and save — the app updates instantly on your phone.

3. Core Components

React Native gives you building blocks that map to native UI elements:

Component	Purpose
`View`	Container/layout, like a `div`
`Text`	Displays text
`Image`	Displays images
`TextInput`	Text entry field
`ScrollView`	Scrollable container
`FlatList`	Efficient scrolling list
`TouchableOpacity` / `Pressable`	Tappable buttons

Example: A Simple Button

```
import { View, Text, Pressable, StyleSheet }
from 'react-native';

export default function App() {
  return (
```

```
    <View style={styles.container}>
      <Pressable
        style={styles.button}
        onPress={() => alert('You tapped me!')}
      >
        <Text style={styles.buttonText}>Tap
Me</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center',
justifyContent: 'center' },
  button: {
    backgroundColor: '#2f95dc',
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 8,
  },
  buttonText: { color: '#fff', fontSize: 16,
fontWeight: '600' },
});
```

4. State: Making Things Interactive

Apps need to remember things — like a counter or form input. React uses the `useState` hook for this.

```
import { useState } from 'react';
import { View, Text, Pressable, StyleSheet }
from 'react-native';

export default function App() {
```

```
  const [count, setCount] = useState(0);

  return (
    <View style={styles.container}>
      <Text style={styles.count}>{count}</Text>
      <Pressable style={styles.button} onPress=
{() => setCount(count + 1)}>
        <Text style={styles.buttonText}>Add
One</Text>
      </Pressable>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, alignItems: 'center',
justifyContent: 'center' },
  count: { fontSize: 48, marginBottom: 20 },
  button: {
    backgroundColor: '#2f95dc',
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 8,
  },
  buttonText: { color: '#fff', fontSize: 16 },
});
```

- `useState(0)` gives you a variable (`count`) starting at `0` and a function (`setCount`) to update it.
- Whenever `setCount` is called, React re-renders the screen automatically.

5. Handling User Input

```
import { useState } from 'react';
import { View, Text, TextInput, StyleSheet }
from 'react-native';

export default function App() {
  const [name, setName] = useState('');

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Enter your name"
        value={name}
        onChangeText={setName}
      />
      <Text style={styles.greeting}>
        {name ? `Hello, ${name}!` : 'Waiting for
your name...'}
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20,
justifyContent: 'center' },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 10,
    marginBottom: 16,
  },
  greeting: { fontSize: 18 },
});
```

6. Displaying Lists

Use `FlatList` for scrollable lists — it's optimized to only render items on screen.

```
import { FlatList, Text, View, StyleSheet } from
'react-native';

const fruits = ['Apple', 'Banana', 'Mango',
'Orange', 'Kiwi'];

export default function App() {
  return (
    <View style={styles.container}>
      <FlatList
        data={fruits}
        keyExtractor={(item) => item}
        renderItem={({ item }) => (
          <View style={styles.item}>
            <Text>{item}</Text>
          </View>
        )}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60,
paddingHorizontal: 20 },
  item: { padding: 15, borderBottomWidth: 1,
borderBottomColor: '#eee' },
});
```

7. Navigating Between Screens

Most apps have multiple screens. The standard tool is **React Navigation**.

```
npx expo install @react-navigation/native
@react-navigation/native-stack
npx expo install react-native-screens react-
native-safe-area-context
```

```
import { NavigationContainer } from '@react-
navigation/native';
import { createNativeStackNavigator } from
'@react-navigation/native-stack';
import { View, Text, Button } from 'react-
native';

const Stack = createNativeStackNavigator();

function HomeScreen({ navigation }) {
  return (
    <View style={{ flex: 1, alignItems:
'center', justifyContent: 'center' }}>
      <Text>Home Screen</Text>
      <Button
        title="Go to Details"
        onPress={() =>
navigation.navigate('Details')}
      />
    </View>
  );
}

function DetailsScreen() {
  return (
    <View style={{ flex: 1, alignItems:
'center', justifyContent: 'center' }}>
      <Text>Details Screen</Text>
```

```
    </View>
  );
}

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component=
{HomeScreen} />
        <Stack.Screen name="Details" component=
{DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

8. Putting It Together: A Mini To-Do App

```
import { useState } from 'react';
import {
  View, Text, TextInput, Pressable, FlatList,
StyleSheet,
} from 'react-native';

export default function App() {
  const [task, setTask] = useState('');
  const [todos, setTodos] = useState([]);

  const addTodo = () => {
    if (task.trim() === '') return;
    setTodos([...todos, { id:
Date.now().toString(), text: task }]);
    setTask('');
  };
```

```
  const removeTodo = (id) => {
    setTodos(todos.filter((t) => t.id !== id));
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>My To-Do
List</Text>
      <View style={styles.row}>
        <TextInput
          style={styles.input}
          placeholder="Add a task..."
          value={task}
          onChangeText={setTask}
        />
        <Pressable style={styles.addButton}
onPress={addTodo}>
          <Text style={styles.addButtonText}>+
</Text>
        </Pressable>
      </View>
      <FlatList
        data={todos}
        keyExtractor={(item) => item.id}
        renderItem={({ item }) => (
          <Pressable style={styles.todoItem}
onPress={() => removeTodo(item.id)}>
            <Text>{item.text}</Text>
            <Text style=
{styles.remove}>✕</Text>
          </Pressable>
        )}
      />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1, paddingTop: 60,
paddingHorizontal: 20 },
  title: { fontSize: 24, fontWeight: 'bold',
marginBottom: 20 },
  row: { flexDirection: 'row', marginBottom: 20
},
  input: {
    flex: 1, borderWidth: 1, borderColor:
'#ccc',
    borderRadius: 8, padding: 10, marginRight:
10,
  },
  addButton: {
    backgroundColor: '#2f95dc', borderRadius: 8,
    paddingHorizontal: 18, justifyContent:
'center',
  },
  addButtonText: { color: '#fff', fontSize: 20
},
  todoItem: {
    flexDirection: 'row', justifyContent:
'space-between',
    padding: 15, borderBottomWidth: 1,
borderBottomColor: '#eee',
  },
  remove: { color: '#e74c3c' },
});
```

Tap the `+` button to add a task. Tap a task to remove it. This one file demonstrates state, input, lists, and styling working together.

Next Steps

- **Styling deeper:** learn Flexbox properties (`flexDirection`, `justifyContent`, `alignItems`) — they control almost all layout.
- **Fetching data:** use `fetch()` or a library like `axios` inside `useEffect` to load data from an API.
- **State management:** for bigger apps, look into Context API or Zustand once `useState` starts feeling limited.
- **Building for real devices:** learn about `eas build` (Expo Application Services) when you're ready to publish to the App Store or Google Play.
- **Official docs:** reactnative.dev and docs.expo.dev are the best references as you grow beyond the basics.

Good luck — the fastest way to learn is to break things and see what happens.